

Bases de Datos a Gran Escala

Sesión 1 · Modelado de datos, modelo relacional y consultas SQL

Diego Sevilla Ruiz

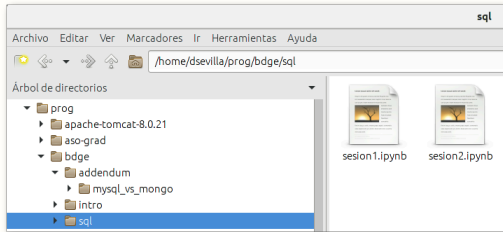
Departamento de Ingeniería y Tecnología de Computadores

Facultad de Informática · Universidad de Murcia

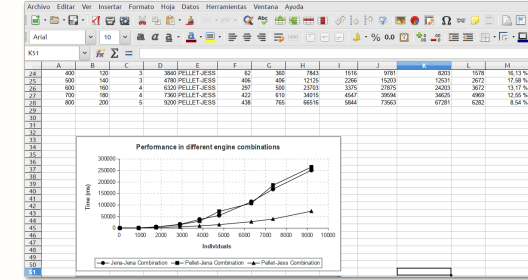
Modelado de datos

Decidir qué son las cosas, qué debe ser cierto y qué consultas deben quedar optimizadas.

Los datos no nacen modelados

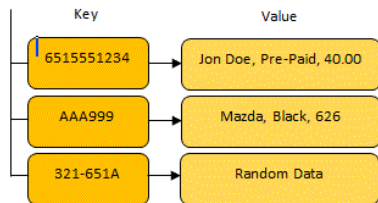


```
Id,PostId,CreationDate,BountyAmount,U
1,1,2015-10-29T00:00:00.000,2
2,2,2015-10-29T00:00:00.000,2
3,3,2015-10-30T00:00:00.000,2
4,5,2015-10-30T00:00:00.000,2
5,7,2015-10-30T00:00:00.000,2
6,8,2015-10-30T00:00:00.000,25.5
7,3,2015-10-30T00:00:00.000,16
8,4,2015-10-30T00:00:00.000,2
9,1,2015-10-30T00:00:00.000,2
```



Directorios y ficheros. CSV y hojas de cálculo. Documentos tabulares.

También pares clave/valor, registros, *streams* y formatos columnares. Llegan sin claves ni tipos garantizados: la estructura no está en los datos, **la imponemos nosotros**.



Tres hechos y ninguna tabla

El 3 de marzo, Ana preguntó cómo ordenar resultados por fecha. Luis respondió al día siguiente y su respuesta recibió 12 votos. Ana la marcó como aceptada.

- ¿Cuántas **entidades** hay aquí? ¿Dos, tres, cinco?
- ¿«Aceptada» es propiedad de la respuesta, de la pregunta o de la relación entre ambas?
- ¿Qué ocurre con la pregunta si Ana borra su cuenta?
- Modelar es decidir esto **antes** de escribir la primera consulta

¿Qué es un modelo de datos?

Todo modelo de datos define tres componentes:

- 1. Estructuras** para albergar los datos
- 2. Restricciones** que deben cumplir
- 3. Operaciones** que se pueden realizar

Ejemplos de los tres componentes

Componente	Preguntas que responde
Estructuras	¿Filas y columnas? ¿Vértices y aristas? ¿Pares clave/valor? ¿Una secuencia de bytes?
Restricciones	¿Todas las filas tienen las mismas columnas? ¿Qué valores admite un atributo? ¿Puede un archivo tener dos padres?
Operaciones	¿Cuál es el valor de la clave <code>x</code> ? ¿Qué filas contienen «plaza»? ¿Cuáles son los siguientes N registros?

Haremos estas tres preguntas a cada sistema del curso.

Los mismos hechos, cuatro modelos

Modelo · sesión	Cómo guarda los hechos	Consultas optimizadas	Consultas costosas
Relacional · 1-2	Users , Posts y una clave ajena	Cualquier pregunta no prevista al diseñar	Reunir una pregunta con todo lo suyo: varios JOIN
Documentos · 3-4	La pregunta, con sus respuestas dentro	Mostrar una pregunta completa de una vez	«¿Qué ha respondido Luis?», salvo duplicar
Familia de columnas · 5-6	Una tabla por consulta; la clave decide dónde vive cada fila	La consulta prevista, a gran escala	Cualquier consulta no prevista
Grafo · 7	Nodos y aristas: Usuario , Pregunta , Respuesta	Recorridos: quién responde a quién	Agregados sobre todo el conjunto

Ninguno es «el correcto». Guardar junto (**agregar**) o guardar aparte y enlazar (**referenciar**) es la decisión que volverá en la sesión 4.

¿Por qué modelar?

- Describe el problema en el **nivel de abstracción adecuado** y descarta lo accesorio
- Decide qué consultas quedan **optimizadas** y cuáles resultan **costosas**: es una decisión de rendimiento, no sólo de orden
- Hace explícitas las reglas del dominio para que el sistema **impida** los estados imposibles
- Permite que varias aplicaciones usen los mismos datos sin negociar cada consulta
- Los datos **sobreviven** a las aplicaciones que los crearon: el modelo es lo que queda cuando cambian el lenguaje, el producto y la moda

Modelo frente a datos crudos



```
-- Schema stackoverflow
--
CREATE SCHEMA IF NOT EXISTS 'stackoverflow' DEFAULT CHARACTER SET latin1;
USE 'stackoverflow';

-- Table 'stackoverflow`.`Users`
--
CREATE TABLE IF NOT EXISTS 'stackoverflow`.`Users` (
  'Id' INT(11) NOT NULL DEFAULT '0',
  'DisplayName' TEXT NULL DEFAULT NULL,
  'Views' INT(11) NULL DEFAULT NULL,
  'DownVotes' INT(11) NULL DEFAULT NULL,
  'LastAccessDate' TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  'ProfileImageUrl' TEXT NULL DEFAULT NULL,
  'WebsiteUrl' TEXT NULL DEFAULT NULL,
  'Reputation' INT(11) NULL DEFAULT NULL,
  'Location' TEXT NULL DEFAULT NULL,
  'Age' INT(11) NULL DEFAULT NULL,
  'UpVotes' INT(11) NULL DEFAULT NULL,
  'CreationDate' TIMESTAMP NOT NULL DEFAULT '0000-00-00 00:00:00',
  'AboutMe' TEXT NULL DEFAULT NULL,
  'AccountId' INT(11) NULL DEFAULT NULL,
  PRIMARY KEY ('Id'),
  ENGINE = InnoDB
  DEFAULT CHARACTER SET = utf8;

-- Table 'stackoverflow`.`Posts`
--
CREATE TABLE IF NOT EXISTS 'stackoverflow`.`Posts` (
  'Id' INT(11) NOT NULL DEFAULT '0',
  'ParentId' INT(11) NULL DEFAULT NULL,
  'Body' TEXT NULL DEFAULT NULL,
  'ViewCount' INT(11) NULL DEFAULT NULL,
  'LastEditorDisplayName' TEXT NULL DEFAULT NULL,
  'ClosedDate' TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  'Title' TEXT NULL DEFAULT NULL,
  'LastEditorUserId' INT(11) NULL DEFAULT NULL,
  'LastActivityDate' TIMESTAMP NOT NULL DEFAULT '0000-00-00 00:00:00',
  'LastEditDate' TIMESTAMP NOT NULL DEFAULT '0000-00-00 00:00:00',
  'AnswerCount' INT(11) NULL DEFAULT NULL,
  'CommentCount' INT(11) NULL DEFAULT NULL,
  'AcceptedAnswerId' INT(11) NULL DEFAULT NULL,
  'Score' INT(11) NULL DEFAULT NULL,
  'OwnerDisplayName' TEXT NULL DEFAULT NULL,
  'PostTypeId' INT(11) NULL DEFAULT NULL,
  'OwnerUserId' INT(11) NULL DEFAULT NULL,
  'Tags' TEXT NULL DEFAULT NULL,
  'CreationDate' TIMESTAMP NOT NULL DEFAULT '0000-00-00 00:00:00',
  'FavoriteCount' INT(11) NULL DEFAULT NULL,
  'CommunityOwnedDate' TIMESTAMP NOT NULL DEFAULT '0000-00-00 00:00:00',
  PRIMARY KEY ('Id'),
  INDEX 'ParentId' ('ParentId') ASC,
  INDEX 'OwnerUserId' ('OwnerUserId') ASC,
  INDEX 'AcceptedAnswerId' ('AcceptedAnswerId') ASC,
  CONSTRAINT 'Posts_ibfk_1'
    FOREIGN KEY ('ParentId')
      REFERENCES 'stackoverflow`.`Posts` ('Id'),
  CONSTRAINT 'Posts_ibfk_2'
    FOREIGN KEY ('OwnerUserId')
      REFERENCES 'stackoverflow`.`Users` ('Id'),
  CONSTRAINT 'Posts_ibfk_3'
    FOREIGN KEY ('AcceptedAnswerId')
      REFERENCES 'stackoverflow`.`Posts` ('Id'),
  ENGINE = InnoDB
  DEFAULT CHARACTER SET = utf8;

-- Table 'stackoverflow`.`Comments`
--
CREATE TABLE IF NOT EXISTS 'stackoverflow`.`Comments` (
  'Id' INT(11) NOT NULL DEFAULT '0',
  'UserId' INT(11) NULL DEFAULT NULL,
  'PostId' INT(11) NULL DEFAULT NULL,
  'Score' INT(11) NULL DEFAULT NULL,
  'CreationDate' TEXT NULL DEFAULT NULL,
  'UserDisplayName' TEXT NULL DEFAULT NULL,
  'Text' TEXT NULL DEFAULT NULL,
  PRIMARY KEY ('Id'),
  INDEX 'PostId' ('PostId') ASC,
  INDEX 'UserId' ('UserId') ASC,
  CONSTRAINT 'Comments_ibfk_1'
    FOREIGN KEY ('PostId')
      REFERENCES 'stackoverflow`.`Posts` ('Id'),
  CONSTRAINT 'Comments_ibfk_2'
    FOREIGN KEY ('UserId')
      REFERENCES 'stackoverflow`.`Users` ('Id'),
  ENGINE = InnoDB
  DEFAULT CHARACTER SET = utf8;
```




«Show me your flowcharts and conceal your tables, and I shall continue to be mystified. Show me your tables, and I won't usually need your flowcharts; they'll be obvious.»

— Fred Brooks, *The Mythical Man-Month*, 1975

Cincuenta años después sigue siendo el mejor resumen de por qué empezamos por el modelo.

El modelo es lo que no se puede delegar

- Una IA escribe hoy SQL, *pipelines* de agregación o Cypher con soltura, y recuerda la sintaxis mejor que nosotros
- No puede decidir qué es una **entidad**, cuándo dos filas son **la misma cosa**, qué debe ser **cierto siempre** ni qué consultas deben quedar **optimizadas**
- Puede proponer un esquema; sólo quien conoce el dominio sabe si es el correcto
- Saber modelar es lo que permite **dirigir** y **criticar** la herramienta en lugar de aceptar su primera respuesta

El esquema es el contexto

- Sobre un esquema que se explica a sí mismo —nombres, tipos, claves, restricciones— una IA acierta casi siempre
- Sobre un `JSON` sin esquema ni documentación produce consultas **verosímiles**: se ejecutan y devuelven filas
- Lo que no puede saber si no está escrito: que `OwnerId` es nulo porque se borró la cuenta, o que hay votos que apuntan a publicaciones que ya no existen
- **Un resultado con filas no es un resultado correcto**: verificaremos lo que devuelve una consulta y lo que cuesta

Modelo relacional

Tablas, claves, restricciones y álgebra para consultar sin depender de cómo se almacenan físicamente los datos.

Modelo relacional

- Propuesto por **E. F. Codd en 1970**
- Modela los datos mediante **relaciones**, habitualmente representadas como tablas
- Se apoya en teoría de conjuntos, lógica de predicados y álgebra relacional
- Su fuerza es la independencia entre los datos y las aplicaciones que los usan: permite formular consultas no previstas al diseñar la base de datos

Conceptos esenciales

- Una relación tiene un **esquema**: atributos con nombre y dominio
- Cada fila o **tupla** sigue ese mismo esquema; cada atributo tiene su propio tipo o dominio
- Una **clave primaria** identifica de forma única cada fila y no admite NULL
- Una **clave ajena** expresa una referencia a una fila de otra tabla
- La normalización busca evitar redundancias y anomalías; la desnormalización es una decisión física posterior y consciente, a la que volveremos en la sesión 4

Clave primaria y clave ajena

Users		Posts		
<u>UserId</u>	Name	<u>PostId</u>	UserId	Title
1	Diego	1	1	SQL Question
38	Pepe	2	38	Java!

`Posts.UserId` referencia `Users.UserId` : no debería existir un *post* cuyo usuario no exista.

Restricciones de integridad

Restricción	Regla
Clave / entidad	No hay dos filas con la misma clave primaria; su valor no es nulo.
Dominio	Cada atributo solo contiene valores de su tipo y reglas asociadas.
Referencial	Una clave ajena apunta a una fila existente, salvo que se permita <code>NULL</code> .
De negocio	Reglas propias del dominio: importe positivo, fecha de fin posterior al inicio, etc.

Álgebra relacional

- Nos permite abstraernos de la representación física
- Las operaciones producen nuevas relaciones: se pueden componer y optimizar
- Operaciones que veremos en SQL: selección, proyección, producto cartesiano, unión y reunión (*join*)

Álgebra relacional

select

project

join

Consultas equivalentes, costes diferentes

El optimizador puede reordenar operaciones equivalentes para reducir el volumen de datos intermedio.

$$\sigma_{p=\text{conoce}}(R) \bowtie_{o=s} \left(\sigma_{p=\text{tieneCuenta}}(R) \bowtie_{o=s} \sigma_{p=\text{web}}(R) \right)$$

$$\left(\sigma_{p=\text{conoce}}(R) \bowtie_{o=s} \sigma_{p=\text{tieneCuenta}}(R) \right) \bowtie_{o=s} \sigma_{p=\text{web}}(R)$$

$$\sigma_{p_1=\text{conoce} \wedge p_2=\text{tieneCuenta} \wedge p_3=\text{web}}(R \times R \times R)$$

No todas tienen el mismo coste, aunque describan el mismo resultado.

SQL: consultar datos

El lenguaje declarativo para expresar qué resultado queremos; el SGBD decide cómo obtenerlo.

SQL y sus dialectos

- SQL es un estándar ANSI/ISO; SQL:2023 incorpora, entre otras extensiones, consultas de grafos de propiedades
- **DML**: consulta y modificación de datos. **DDL**: definición del esquema
- SQL trabaja con tablas, columnas y filas; a diferencia del álgebra relacional, una consulta puede devolver duplicados (*bag semantics*)
- En esta sesión usamos el núcleo común de SQL. Cuando aparezca una diferencia, se indicará el motor: **SQLite**, **MySQL** o **DuckDB**

Herramientas de la asignatura

Herramienta	Papel
SQLite	Primer contacto: un único fichero, consola sencilla y sin servidor.
MySQL	Prácticas con servidor, transacciones, índices y planes <code>EXPLAIN</code> .
DuckDB	Análisis de datos y lectura directa de Parquet; útil para contrastar optimizaciones analíticas.

Los ejemplos deben ejecutarse con el motor indicado: la sintaxis y, sobre todo, los planes no son idénticos.

Modo de trabajo

- El objetivo es manejar SQL con comodidad, no memorizar un catálogo de funciones
- Trabajaremos sobre una instantánea de datos de Stack Overflow cargada por los notebooks de la asignatura
- El mismo conjunto permite estudiar esquema, consultas, calidad de datos, relaciones, agregación y rendimiento
- El cuaderno prepara los datos desde Parquet; DuckDB puede consultarlos directamente y MySQL permite estudiarlos como base de datos de servidor

Primer contacto con SQLite

SQLite almacena una base de datos relacional en un fichero y añade comandos de administración al intérprete:

```
$ sqlite3 ejemplo.db
sqlite> .tables
sqlite> .schema Posts
sqlite> .open otro-fichero.db
sqlite> .help
```

Estos comandos no son SQL: son ayudas de la consola de SQLite.

Base de datos de Stack Overflow

Questions

Tags

Users

Badges

Unanswered

Ask Question

How to remove comma at end of last line in an array

0

▲▼★

I am using `sed '$s/,,$//'` to try to remove the comma at the end of the final line of an array, which has the opening and closing brackets on their lines (as shown below). This command `sed '$s/,,$//'` isn't working as intended.

If I do `sed '$s/,$//'` it will remove the closing array bracket, so I know I'm kind of on the right track....

```
[{"foo": "foofoo", "fuzz": ["fee, fi, fo, fum"]}, {"bar": "barbar", "fuzz": ["blah", "blah"]}]
```

this is for a bash script. please do not provide answer that uses perl.

bash

sed

share

edit

close

flag

edited 1 hour ago

asked 1 hour ago

Leahcim

8,207 ● 27 ● 100 ● 196

Is this supposed to be valid JSON? The thing producing it should be fixed to stop adding the incorrect comma. – chepner 1 hour ago

@chepner yes, it is. not sure how to do that (subject for another question...) – Leahcim 1 hour ago

add a comment

2 Answers

active

oldest

votes

▲▼★

1

There is no comma in last line so `'$s/,,$//'` will not work for `sed`.

Instead of `sed` you can use `awk`:

```
awk ' $1 == "]" {sub(/,$/, "", s)} NR>1 {print s} {s=$0} END{print s}' file
```

```
[{"foo": "foofoo", "fuzz": ["fee, fi, fo, fum"]}, {"bar": "barbar", "fuzz": ["blah", "blah"]}]
```

asked today

viewed 16 times

active today

Featured on Meta

New Navigation Release Candidate

Hot Meta Posts

8

Is it wrong to try to draw attention to a post I feel should be closed?

5

What if I think a deleted answer is worthwhile?

25

Syntax Highlighting for Neo4j / Cypher questions

Looking for a job?

Senior PHP Developer

Trovit

Barcelona, Spain

php

mysql

Systems Engineer

Brandcrumb

Barcelona, Spain

hadoop

nosql

Related

190

How can I remove the first

Exploración de datos en Stack Exchange Data Explorer



Esquema de Stack Overflow

- **Posts:** preguntas y respuestas; `PostTypeId` distingue ambos tipos y `ParentId` enlaza una respuesta con su pregunta
- **Users:** usuarios y reputación
- **Comments** y **Votes:** actividad relacionada con un *post* y, cuando procede, un usuario
- **Tags** y **PostTags:** relación muchos-a-muchos entre preguntas y etiquetas
- En algunas instantáneas existe además una columna de etiquetas textual por motivos de rendimiento: es un ejemplo de desnormalización

Diagrama del esquema



El esquema completo de Stack Exchange

SEDE incluye muchas más tablas que las cinco que usaremos en prácticas.

Fuente: «The Schema Visualized».



El subconjunto que usaremos en prácticas

Las cinco tablas publicadas, con sus claves ajenas reales.



Lo que el diagrama no dice

- `Posts` se referencia a sí misma dos veces: `ParentId` enlaza una respuesta con su pregunta y `AcceptedAnswerId` la pregunta con su respuesta aceptada. Aparecen como claves ajenas, pero no se dibujan para no enredar el diagrama
- Casi todas las claves ajenas son **opcionales**: un *post* puede no tener usuario propietario porque la cuenta se borró, y un voto puede ser anónimo
- **Que una clave ajena esté declarada no significa que se cumpla**: este *dump* contiene votos y comentarios que apuntan a publicaciones ya borradas. Habrá que decidir explícitamente qué hacer con ellos antes de activar las restricciones
- El diagrama no está dibujado a mano: se genera a partir de los metadatos del propio esquema Parquet, los mismos que en la sesión 0 generaban el `CREATE TABLE`

SELECT: forma general

```
SELECT lista_de_columnas  
FROM lista_de_tablas  
WHERE condicion;
```

Primero se seleccionan las filas que cumplen la condición y después se proyectan las columnas solicitadas.

Elegir columnas

```
SELECT * FROM Tags;
```

```
SELECT Id, Title FROM Posts;
```

- `*` es útil para explorar, pero en consultas reales conviene pedir solo las columnas necesarias
- Las comillas de identificadores no son portables: evitamos depender de ellas usando nombres sencillos y sin palabras reservadas

DISTINCT, alias y LIMIT

```
SELECT DISTINCT PostTypeId FROM Posts;
```

```
SELECT DisplayName AS nombre FROM Users;
```

```
SELECT DisplayName FROM Users LIMIT 2;
```

- **DISTINCT** elimina duplicados del resultado
- **AS** nombra columnas o expresiones
- **LIMIT** facilita la exploración; no sustituye a un orden explícito

Ordenar resultados

```
SELECT Id, DisplayName, Reputation  
FROM Users  
ORDER BY Reputation DESC, Id ASC;
```

Sin `ORDER BY`, SQL **no garantiza** ningún orden de salida.

Selección y proyección

Álgebra relacional	SQL
Selección de filas: $\sigma_c(R)$	SELECT * FROM R WHERE c
Proyección de columnas: $\Pi_{a,b}(R)$	SELECT a, b FROM R

```
SELECT Id, DisplayName
FROM Users
WHERE Reputation > 100;
```

El optimizador puede adelantar proyecciones, pero debe conservar las columnas necesarias para evaluar los filtros y las uniones.

Filtrar con WHERE

```
SELECT Id, DisplayName, Reputation
FROM Users
WHERE DisplayName = 'Elenasys';
```

Operador	Uso
= , <> , < , >=	Comparaciones
BETWEEN ... AND ...	Rango inclusivo
IN (...)	Pertenencia a una lista
LIKE	Patrón de texto
NOT , AND , OR	Composición lógica

Patrones y condiciones compuestas

```
SELECT DisplayName
FROM Users
WHERE Reputation > 100
      AND Location LIKE '%Spain%';
```

```
SELECT DisplayName
FROM Users
WHERE DisplayName LIKE '%sys%';
```

`AND` tiene mayor prioridad que `OR` ; usar paréntesis hace la intención inequívoca.

Expresiones y alias

```
SELECT DisplayName,  
       UpVotes + DownVotes AS votos_totales  
FROM Users  
WHERE UpVotes + DownVotes > 10;
```

El alias de `SELECT` no se puede usar de forma portable en `WHERE` : en esta fase de la consulta todavía no existe. Para reutilizarlo, se usa una subconsulta, una CTE o se repite la expresión.

NULL no es un valor ordinario

- NULL significa desconocido, ausente o no aplicable; no equivale a 0 ni a una cadena vacía
- col = NULL no encuentra filas: se usa col IS NULL o col IS NOT NULL
- Las comparaciones con NULL producen «desconocido», que WHERE no conserva

```
SELECT Id, ParentId  
FROM Posts  
WHERE ParentId IS NULL;
```

Funciones de agregación

```
SELECT AVG(Score) AS puntuacion_media,  
       MAX(Score) AS puntuacion_maxima,  
       COUNT(*) AS numero_posts  
FROM Posts;
```

- `COUNT(*)` cuenta filas; `COUNT(columna)` solo cuenta valores no nulos
- `AVG`, `SUM`, `MIN` y `MAX` ignoran los valores nulos
- `COUNT(DISTINCT OwnerUserId)` cuenta valores distintos no nulos

Funciones: núcleo y dialectos

Familia	Ejemplos portables o frecuentes
Numéricas	ABS , CEIL , POWER , SQRT
Cadenas	LOWER , UPPER , TRIM , SUBSTR , REPLACE
Control de nulos	COALESCE , NULLIF
Condicionales	CASE WHEN ... THEN ... ELSE ... END
Tipos	CAST(expr AS tipo)

CONCAT , funciones de desviación típica, aleatoriedad y fechas presentan diferencias entre motores; el notebook especifica las variantes necesarias.

CASE y COALESCE

```
SELECT DisplayName,  
       CASE  
         WHEN Reputation > 100 THEN 'Veterano'  
         ELSE 'Rookie'  
       END AS tipo,  
       COALESCE(UpVotes, 0) AS votos_positivos  
FROM Users;
```

COALESCE(a, b, ...) devuelve el primer argumento que no es nulo.

Fechas: filtrar sin perder el índice

Evitar, cuando se busca rendimiento:

```
WHERE YEAR(CreationDate) = 2025
```

Preferir un rango semicerrado:

```
WHERE CreationDate >= '2025-01-01'  
      AND CreationDate < '2026-01-01'
```

Las funciones y literales de fechas dependen del motor; el principio de no envolver la columna del filtro es general.

Consultar varias tablas

Producto cartesiano, `JOIN`, agrupación y subconsultas.

Producto cartesiano y equi-join

El producto cartesiano combina cada fila de R_1 con cada fila de R_2 . Rara vez es el resultado final deseado.

$$R_1 \bowtie_{\text{Uid}_r=\text{Uid}} R_2 = \sigma_{\text{Uid}_r=\text{Uid}}(R_1 \times R_2)$$

Un **equi-join** conserva las combinaciones cuyas claves relacionadas coinciden.

JOIN explícito

```
SELECT c.Id, p.Title
FROM Comments AS c
JOIN Posts AS p ON p.Id = c.PostId
WHERE p.Score > 10;
```

- El `JOIN ... ON` hace visible la condición de unión
- Evitamos la sintaxis antigua con tablas separadas por comas: es más fácil olvidar una condición y crear un producto cartesiano accidental

USING: cuando la columna se llama igual

```
SELECT PostId, c.UserId, v.VoteTypeId  
FROM Comments AS c  
JOIN Votes AS v USING (PostId);
```

- USING (col) abrevia ON a.col = b.col, y sólo vale si la columna **se llama igual** en las dos tablas
- La columna aparece **una vez** en el resultado y se cita sin cualificar; con ON saldría dos veces y SELECT PostId sería ambiguo
- Si los nombres difieren, como Posts.Id frente a Comments.PostId, hay que seguir usando ON

Es la misma reunión, escrita sin repetir el nombre de la clave

Preguntas y quien las hizo

```
SELECT u.DisplayName, p.Title
FROM Posts AS p
JOIN Users AS u ON u.Id = p.OwnerUserId
WHERE p.PostTypeId = 1;
```

La misma idea se reutiliza para enlazar comentarios, votos, etiquetas y el historial con los *posts*.

LEFT JOIN: conservar filas sin coincidencia

```
SELECT p.Id, COUNT(a.Id) AS respuestas
FROM Posts AS p
LEFT JOIN Posts AS a
  ON a.ParentId = p.Id
  AND a.PostTypeId = 2
WHERE p.PostTypeId = 1
GROUP BY p.Id;
```

Las preguntas sin respuestas siguen apareciendo, con 0 respuestas. Con un JOIN normal desaparecerían.

GROUP BY y HAVING

```
SELECT OwnerUserId, PostTypeId, COUNT(*) AS numero_posts
FROM Posts
GROUP BY OwnerUserId, PostTypeId
HAVING COUNT(*) > 10
ORDER BY OwnerUserId ASC, numero_posts DESC;
```

- WHERE filtra filas **antes** de agrupar
- HAVING filtra grupos **después** de agregar
- En SQL estricto, cada expresión seleccionada debe estar agrupada, agregada o ser funcionalmente dependiente de las columnas agrupadas

Agregación con nombres

```
SELECT u.DisplayName, COUNT(*) AS respuestas
FROM Posts AS p
JOIN Users AS u ON u.Id = p.OwnerUserId
WHERE p.PostTypeId = 2
GROUP BY u.Id, u.DisplayName
HAVING COUNT(*) > 100
ORDER BY respuestas DESC;
```

La consulta explica una pregunta concreta: «¿qué usuarios han aportado muchas respuestas?».

- `u.DisplayName` no añade un criterio de agrupación: depende funcionalmente de `u.Id`, clave primaria, y hay un único nombre por usuario
- MySQL 8 detecta esa dependencia y acepta `GROUP BY u.Id` a secas; listar la columna es la forma portable y la que exige SQL estricto cuando no se agrupa por una clave

Tablas como conjuntos

```
SELECT UserId FROM Comments  
UNION  
SELECT OwnerUserId FROM Posts;
```

- `UNION` elimina duplicados; `UNION ALL` los conserva
- `INTERSECT` y `EXCEPT` están disponibles en muchos motores, con diferencias de versión
- `IN` expresa pertenencia a un conjunto; con `NULL` hay que revisar cuidadosamente su semántica

UNION ALL : apilar hechos, no deduplicarlos

```
SELECT 'pregunta' AS Tipo, Id, OwnerUserId, CreationDate
FROM Posts WHERE PostTypeId = 1
UNION ALL
SELECT 'respuesta', Id, OwnerUserId, CreationDate
FROM Posts WHERE PostTypeId = 2;
```

- UNION tiene que ordenar o construir una estructura auxiliar para descartar duplicados; UNION ALL no
- Cuando las ramas son disjuntas por construcción, UNION ALL es a la vez lo correcto y lo barato
- Todas las ramas necesitan el mismo número de columnas y tipos compatibles; los nombres los pone la primera
- Cada rama aporta su tipo como columna constante: eso es lo que distingue después a cada fila

El riesgo: identificadores que chocan

```
SELECT 'alta_usuario' AS Tipo, Id FROM Users
UNION ALL
SELECT 'pregunta', Id FROM Posts WHERE PostTypeId = 1;
```

- `Users.Id` y `Posts.Id` son espacios de identificadores independientes, y ambos empiezan en 1
- El usuario 5 y la pregunta 5 son hechos distintos que comparten el valor de `Id`
- `PRIMARY KEY (Id)` fallaría: la clave tiene que ser `(Tipo, Id)`
- Preguntas y respuestas, en cambio, salen ambas de `Posts` y nunca chocan entre sí

Una tabla de hechos con UNION ALL

```
INSERT INTO Hechos (Tipo, EntidadId, UserId, CreationDate)
SELECT 'alta_usuario', Id, Id, CreationDate FROM Users
UNION ALL
SELECT 'pregunta', Id, OwnerUserId, CreationDate
FROM Posts WHERE PostTypeId = 1
UNION ALL
SELECT 'respuesta', Id, OwnerUserId, CreationDate
FROM Posts WHERE PostTypeId = 2;
```

- Una tabla de hechos reúne en una sola línea de tiempo eventos de origen distinto
- PRIMARY KEY (Tipo, EntidadId) es lo que impide que el alta del usuario 5 y la pregunta 5 colisionen
- Ordenar por CreationDate responde preguntas que ninguna tabla de origen contesta por sí sola

Subconsultas y tablas derivadas

```
SELECT u.DisplayName, c.numero_comentarios
FROM Users AS u
JOIN (
    SELECT UserId, COUNT(*) AS numero_comentarios
    FROM Comments
    GROUP BY UserId
) AS c ON c.UserId = u.Id;
```

Una subconsulta en `FROM` produce una tabla derivada que se puede unir y consultar como las demás.

Consultas correlacionadas

```
SELECT u.Id, u.DisplayName
FROM Users AS u
WHERE 100 < (
    SELECT COUNT(*)
    FROM Posts AS p
    WHERE p.OwnerUserId = u.Id
    AND p.PostTypeId = 1
);
```

La subconsulta depende de la fila externa. Un optimizador puede transformarla, pero debemos comprobar el plan y los índices antes de asumir que tendrá un buen coste.

Promedio de respuestas: una sutileza de **NULL**

```
SELECT AVG(COALESCE(a.respuestas, 0)) AS media_respuestas
FROM Posts AS p
LEFT JOIN (
    SELECT ParentId, COUNT(*) AS respuestas
    FROM Posts
    WHERE PostTypeId = 2
    GROUP BY ParentId
) AS a ON a.ParentId = p.Id
WHERE p.PostTypeId = 1;
```

Sin **COALESCE**, **AVG** ignora las preguntas sin respuesta y la media queda sesgada al alza.

Consultas correctas y eficientes

El mismo resultado lógico puede requerir cantidades de trabajo muy distintas.

Leer un plan de MySQL

```
EXPLAIN
SELECT PostId
FROM Votes
WHERE PostId NOT IN (SELECT Id FROM Posts);
```

Empezaremos por preguntas simples:

- ¿Se recorre toda la tabla o se usa un índice?
- ¿Cuántas filas estima el optimizador?
- ¿Qué unión y qué orden de acceso ha elegido?

`EXPLAIN ANALYZE` añade medidas reales y ejecuta la consulta.

Ideas que conectan con BDGE

- Elegir solo las columnas necesarias evita transferencia y lectura innecesarias
- Filtrar pronto permite *predicate pushdown* en motores columnares y ficheros Parquet
- Los índices aceleran ciertos accesos, pero aumentan el coste de inserciones y actualizaciones
- Las estadísticas, la distribución de los datos y la cardinalidad determinan el plan elegido
- La desnormalización puede acelerar lecturas, a cambio de consistencia y coste de actualización

Para continuar

- Definir esquemas con DDL y restricciones
- Insertar, actualizar y borrar datos de forma segura
- Diseñar índices y contrastar planes con `EXPLAIN`
- Trabajar con JSON y decidir qué parte debe seguir siendo relacional